
VECTADB: AN ONTOLOGY-NATIVE VECTOR AND GRAPH DATABASE FOR AUDITABLE AI SYSTEMS

Roberto Williams Batista
Independent Researcher
robertobatista@ieee.org

*

ABSTRACT

The use of Large Language Models (LLMs) and agentic systems applications increased exponentially in areas with operational and legal liability, such as health, financial, legal, and government. Concurrently, the AI systems' architectures became more complex with the incremental addition of elements such as tools, memory, protocols, guardrails, and multi-step decisions to accomplish a diverse quantity of tasks, demanding the audit systems' capability to also grow in complexity to perform verification of such systems. The analysis surpasses the use of system logs, providing the ability to reconstruct the AI system rationale in the steps taken, including data, tools, and their relationship with policies, controls, and regulatory obligations.

This paper introduces VectaDB, an ontology-native metadatabase that unifies vector embeddings, graph structure, and typed ontologies to provide an audit-first substrate for AI applications, offering a robust analytical abstraction layer for AI auditing. VectaDB stores (i) the similarity search semantic representations, (ii) explicit relationships for provenance, causal, and lineage queries, and (iii) an ontological layer for constraints and interpretability.

VectaDB enables symbolic and semantic investigations due to the created representation of prompts, tool calls, retrieved evidence, artifacts, and outputs, linking the objects by provenance relations, retrieving previous similar incidents, tracing decision lineage, validating data quality constraints, and producing compliance evidence.

The current open-source implementation uses SurrealDB for graph and multi-model storage [1] and Qdrant for vector indexing and similarity search [2], supporting hybrid retrieval and audit queries suitable for Retrieval-Augmented Generation (RAG) pipelines and memory augmented agents. It also uses the Rust programming language for system performance and Python for test scripts.

Keywords vector database; graph database; ontology; knowledge graph; provenance; AI audit; LLM observability; agentic systems; RAG

1 Introduction

All the rapidly increasing complexity in AI systems and their adoption has made it difficult for society, government, industry, and the research community to understand the full extent of ethical, economic, and behavioral impacts on humans and their environment. The AI regulations are, at large, incipient, incomplete, and highly volatile, requiring constant review and a suitable audit ecosystem of tools to enforce the necessary guidelines. VectaDB was envisioned to contribute to the AI audit ecosystem, providing means to audit AI Agentic Systems to answer practical questions as: (i) What information was retrieved or observed before a decision? (ii) Which tool calls were called, and what were their inputs and outputs? (iii) Which prior conversations, memories, or documents influenced the response? (iv) Can we reproduce and explain a system's behavior when its reported harm or regulation demands evidence? Usual system telemetry and logging captures data events, but typically lacks a shared semantic model that supports evidence-centric

*Open-source project (Apache License 2.0): GitHub repository available at <https://github.com/robertowilliams/VectaDB>

queries and reconstruction enablement. VectaDB addresses this gap by combining three paradigms: (1) vector search for semantic similarity and approximate recall, (2) graph structure for explicit relationships, lineage, and multi-level traversal, and (3) ontologies to type entities and relations, validate constraints, and increase interpretability. The resulting system can store both the raw artifacts (e.g., prompts, retrieved chunks, tool outputs) and their semantic/structural context, enabling audit workflows that combine similarity search, graph traversal, and schema-based validation. Intended for teams building RAG pipelines and agentic automation systems who need investigation-grade traces.

1.1 Audit-first motivation and design goals

The auditing process for LLMs and Agentic Systems is, in essence, different from software due to their probabilistic nature, dynamic retrieval corpora, and external tool execution, contrasting with the deterministic profile of programming. VectaDB was designed to fulfill common audit requirements in such probabilistic environment providing: it is a data and query abstraction layer intended to be integrated into application stacks such as automation agents, such data architecture enables a comprehensive inspection of the events and actions over time, supporting the following use cases:

- Trace reconstruction: creates a graph representation of events and artifacts taken by the agent, providing a queryable interface.
- Evidence traceability: registers and fetches documents, memories, tools’ outputs, involved in the agent action process.
- Semantic incident response: retrieves previous semantically similar incidents such as prompts, outputs, and policy violations.
- Schema and policy alignment: enforces schema in trace ingestion, warranting the presence of required fields like model identifiers, task IDs, timestamps, and data sources. Also supporting queries that map the compliance control as retention, data minimization, and access control.
- Reproducibility and accountability: supports process replay using stored prompts, retrieval parameters, and tool call inputs and outputs.

1.2 Contributions

This paper gives the following key contributions:

- Establishes an ontology-native data model to unify vectors, graphs, and typed entities to represent AI traces and knowledge in a single underlayer.
- Introduces an audit-oriented data event ingestion and query model, supporting symbolic lineage queries and semantic similarity inspections.
- Presents the relevance of hybrid retrieval and graph traversal for supporting agent memory, decision provenance, and generation of compliance evidence.
- An open-source implementation (Apache 2.0) built on SurrealDB and Qdrant DB to support graph and vector data store correspondingly, comprising guidance for integrating embeddings, RAG pipelines, and observability tooling.

2 Background and related work

VectaDB integrates three established research areas: vector similarity search, graph representation, and semantic-web ontologies. It addresses the emerging technical requirement for auditable LLMs and agentic systems.

2.1 LLMs, tool use, and agentic systems

Early mass-adoption LLMs such as GPT-3 [3] and GPT-4 [4] have enabled general-purpose natural-language interfaces; however, their behavior highly depends on context, such as instructions, examples, present text, and external actions (tool executions and corpora retrieval within the agent loop [5, 6, 7]). To address LLM limitations multiple different elements were developed and integrated to expand and improve LLM applications. RAG [5] exposes knowledge to searchable indexes, improving factuality, drastically reducing model hallucinations [8], and enabling systems to cite precise evidence grounded on source documents. This result is reached because the RAG limits model’s reliance on

unconstrained internal recall and grounds responses in verifiable sources more commonly represented in vector format. Graph-based variants such as GraphRAG [9] build a hierarchical graph index to generate summaries and perform community detection, resulting in a thematic partitioning of the data. Agentic systems extend LLMs with reasoning, planning, coordination, actions, learning, and tool calls [10]. Among the techniques used in agentic system we can cite ReAct [6] interleaves reasoning and acting, improving interpretability via explicit action traces; Toolformer [7] explores self-supervised learning for deciding when to call external tools and Reflexion [11] and Generative Agents [12] emphasize memory and reflection as key components of long-horizon behavior. Surveys of LLM-based autonomous agents [13] highlight that memory, planning, and tool execution are central to agent construction, and their auditability.

2.2 Vector databases and approximate nearest neighbor search

Vector databases deploy approximate nearest neighbor (ANN) search over embedding vectors, a high dimension representation. Libraries such as FAISS [14] and systems such as Milvus [15] popularized high-performance vector indexing and retrieval. Common ANN approaches include hierarchical navigable small world (HNSW), graphs [16] and optimized search pipelines such as ScaNN [17]. Production vector database products and projects provide important additional features such as metadata filtering, hybrid ranking, and operational tooling. Examples include Qdrant, which combines HNSW-based similarity search with structured payload filtering [2]; Weaviate, which supports hybrid search combining vector similarity and BM25 keyword retrieval [18]; Pinecone’s managed service with metadata filtering and hybrid dense and sparse retrieval workflows [19]; and Chroma, a versatile open-source database oriented toward LLM application development with vector and full-text search capabilities, but not suitable for high scale systems [20].

2.3 Graph databases and multi-model databases

Graph databases represent entities and relationships explicitly, enabling multi-hop traversal and causal/lineage analysis. Neo4j popularized the property-graph model [21] and has introduced native vector indexes for similarity search and hybrid analytical queries [22]. JanusGraph targets very large distributed graphs and uses the Gremlin traversal language from Apache TinkerPop [23, 24]. Multi-model databases such as SurrealDB unify document, relational, graph, and vector capabilities under a single query language and storage engine [1].

2.4 Ontologies, knowledge graphs, and provenance standards

Ontologies add machine-interpretable semantics to graphs through shared vocabularies, typing, and constraints. RDF provides the foundation data model for representing labeled graphs as triples [25], OWL extends RDF with richer ontology constructs and formal semantics [26], and SPARQL provides a standardized query language for RDF graphs [27]. SHACL enables validation of RDF graphs against declarative constraints [28]. Provenance standards are directly relevant to auditing AI systems. The W3C PROV family provides a conceptual data model for representing provenance information (entities, activities, agents, and their relations) [29]. Frameworks such as Apache Jena provide open-source infrastructure for RDF and SPARQL [30], and platforms such as Stardog provide enterprise knowledge-graph capabilities including inference and data-quality tooling [31].

2.5 Auditability, transparency, and observability for AI systems

Auditing frameworks for machine learning emphasize documentation and transparency. Model Cards [32] propose standardized reporting for trained models, and Datasheets for Datasets [33] propose analogous documentation for datasets. For LLM evaluation and transparency, HELM provides a broad benchmarking framework and taxonomy of scenarios and metrics [34]. From a regulatory and governance perspective, organizations operating in the EU increasingly consider obligations under the EU Artificial Intelligence Act [35] and data-protection requirements under the GDPR [36]. Although VectaDB is not a compliance solution by itself, it is designed to store the evidence needed to support compliance processes. Operationally, LLM applications benefit from observability infrastructure. LangSmith provides tracing and execution visibility for LLM applications and agents [37], while OpenTelemetry provides a vendor-neutral standard for generating and exporting telemetry such as traces, metrics, and logs [38]. MLflow addresses lifecycle tracking for machine-learning experiments, deployments, and their artifacts [39]. VectaDB is intended to complement these systems by providing a semantically rich, queryable data store for audit traces. Finally, explainability methods such as LIME [40] and SHAP [41] highlight the broader need for interpretable evidence when model decisions affect stakeholders. In VectaDB’s context, the primary focus is on traceability and provenance of AI behaviors rather than feature-attribution for a single prediction.

2.6 Memory layers and hybrid retrieval for agents

Persistent memory layers for agents are an active engineering area. Mem0 positions itself as a universal memory layer for AI agents [42], and Cognee combines vector search with graph databases to create persistent, connected memory for agent applications [43]. At the application-framework level, LangChain provides memory abstractions such as ConversationBufferMemory, which stores conversation history for use in subsequent turns [44]. Hybrid retrieval techniques commonly combine multiple rankings; reciprocal rank fusion (RRF) is a simple, robust approach for combining ranked lists from different retrieval methods [45]. VectaDB incorporates these ideas by treating memory items as typed objects connected by explicit relations, while also supporting semantic similarity search and hybrid ranking strategies.

2.7 What’s New vs Existing Systems

The preceding review reveals that while individual components of VectaDB’s functionality exist across disparate systems, no existing solution provides their unified integration under an audit-first design philosophy. Table 1 summarizes the key differentiators.

Table 1: Comparison of VectaDB with existing database and observability systems across key audit-relevant capabilities.

Capability	VectaDB	Vector DBs (Qdrant, Milvus)	Graph DBs (Neo4j)	Ontology (Jena, Stardog)	Memory (Mem0, Cognee)	Observability (LangSmith)
Vector similarity search	✓	✓	Partial ^a	×	✓	×
Graph traversal	✓	×	✓	✓	Partial ^b	×
Ontology-native typing	✓	×	×	✓	×	×
Typed relations with semantics ^c	✓	×	×	✓	×	×
Ontology-governed hybrid queries	✓	×	×	×	×	×
Audit-first provenance	✓	×	×	Partial ^d	×	Partial ^e
Schema validation at ingestion	✓	×	Partial ^f	✓	×	×
Semantic incident retrieval	✓	×	×	×	×	×
Agent memory with lineage	✓	×	×	×	Partial ^g	×

^aNeo4j added vector indexes in 2023 but vectors remain secondary to the graph model.

^bCognee supports graph structures but lacks typed ontological constraints.

^cIncludes transitive, symmetric, functional, and inverse relation properties.

^dPROV-O ontology can model provenance but requires manual integration.

^eLangSmith captures traces but lacks semantic search and graph queries.

^fNeo4j supports property constraints but not ontology-driven type hierarchies.

^gMemory systems store context but lack explicit provenance links to runs.

2.7.1 Unified Data Model vs. Fragmented Architectures

Existing approaches force application-level orchestration across heterogeneous systems. A typical auditable RAG pipeline might require Qdrant for similarity search, Neo4j for provenance graphs, a separate ontology component for type expansion, and LangSmith for trace capture. VectaDB consolidates these concerns into a single data abstraction where every stored element is represented as a typed entity in a unified trace graph and may additionally carry a vector embedding. This consolidation can reduce cross-system coordination and query fan-out, simplifying trace reconstruction and enabling provenance links to be recorded at the same time as the corresponding artifacts.

2.7.2 Ontology as First-Class Primitive

While knowledge graph systems like Apache Jena [30] and Stardog [31] provide robust ontological reasoning, they lack efficient vector similarity search and were not designed for the high-throughput, low-latency requirements of AI agent observability. Conversely, vector databases treat metadata as secondary filters rather than semantically meaningful types. VectaDB adopts ontology as a foundational layer which entity types define property schemas with cardinality constraints (One, ZeroOrOne, Many, OneOrMore), relation types specify domain/range restrictions along with semantic properties (transitivity, symmetry, functionality), and the OntologyReasoner expands queries at runtime to include subtypes and inferred relations. This design enables queries such as “find all ToolCalls (including subtypes WebSearchCall, DatabaseQueryCall) that influenced outputs flagged by PolicyCheck failures” without requiring application code to enumerate the type hierarchy.

2.7.3 Audit-First Design Philosophy

Observability tools like LangSmith [37] and OpenTelemetry [38] excel at capturing execution traces but store them as unstructured or loosely-structured logs. Reconstructing an audit trail requires post-hoc parsing and manual correlation. VectaDB inverts this paradigm, the trace schema (Table 2) enforces that every Run, Prompt, ToolCall, and ModelOutput is stored with required audit fields (timestamps, identifiers, content hashes) and linked via explicit provenance edges (usedEvidence, produced, evaluatedArtifact). This structure enables the four-query incident investigation pattern demonstrated in Section 6.4: provenance queries identify what evidence was used, causality queries trace root causes, scope queries assess blast radius, and control-evidence queries evaluate whether safeguards functioned.

2.7.4 Hybrid Retrieval with Semantic Reasoning

Memory systems such as Mem0 [42] and Cognee [43] support vector-based retrieval over agent memories, and Cognee additionally incorporates graph structures. However, these systems operate as application-layer orchestration rather than unified query engines. VectaDB’s QueryCoordinator implements a staged execution model that (1) expands entity and relation types using ontological reasoning, (2) executes vector and graph queries in parallel, and (3) merges results using configurable strategies (Union, Intersection, RankFusion, VectorPriority, GraphPriority). Reciprocal Rank Fusion [45] enables principled combination of semantic similarity scores with graph-derived relevance without requiring score normalization across modalities.

2.7.5 Memory as Auditable, First-Class Objects

LangChain’s ConversationBufferMemory [44] and similar abstractions treat memory as opaque context buffers. When an agent’s behavior is questioned, there is no mechanism to determine when a memory was created, why it was created, or which runs accessed it. VectaDB treats MemoryItems as typed entities with full provenance which each memory stores its creation timestamp, embedding for semantic retrieval, memory type classification (episodic, semantic, procedural), and explicit graph relations (createdByRun, accessedByRun, supersedes). This design transforms agent memory from a hidden internal state into an inspectable, versionable, and queryable audit artifact.

2.7.6 Schema Enforcement for Trace Quality

A common challenge in AI auditing is incomplete or inconsistent trace data. VectaDB addresses this through SHACL-inspired [28] validation at ingestion time. The ontology layer defines which fields are required for each entity type, what constraints apply (value ranges, enumerated values, string patterns), and which relations must exist. Traces that fail validation are rejected rather than stored in degraded form, ensuring that audit queries operate over complete, well-formed data.

2.7.7 Open Architecture for Interoperability

Unlike proprietary observability platforms, VectaDB’s open-source implementation (Apache 2.0) exposes a documented data model aligned with W3C PROV concepts [29]. This enables to export to RDF/OWL toolchains for organizations standardizing on semantic-web stacks, integration with existing observability pipelines via the event ingestion API, and customization of ontology modules for domain-specific audit requirements (healthcare, finance, legal). The pluggable embedding architecture supports both cloud providers (OpenAI, Cohere, Voyage AI) and local models, preserving availability and data sovereignty when needed.

In summary, VectaDB’s novelty lies not in any single capability but in their principled integration under an audit-first design philosophy that treats provenance, type safety, and hybrid retrieval as co-equal concerns rather than afterthoughts.

3 VectaDB system design

3.1 Conceptual model

VectaDB’s core abstraction is a unified knowledge substrate in which every stored element is simultaneously:

- an object with typed attributes (for structured queries),
- a node in a graph (for lineage and causal traversal), and optionally
- a vector embedding (for semantic similarity search).

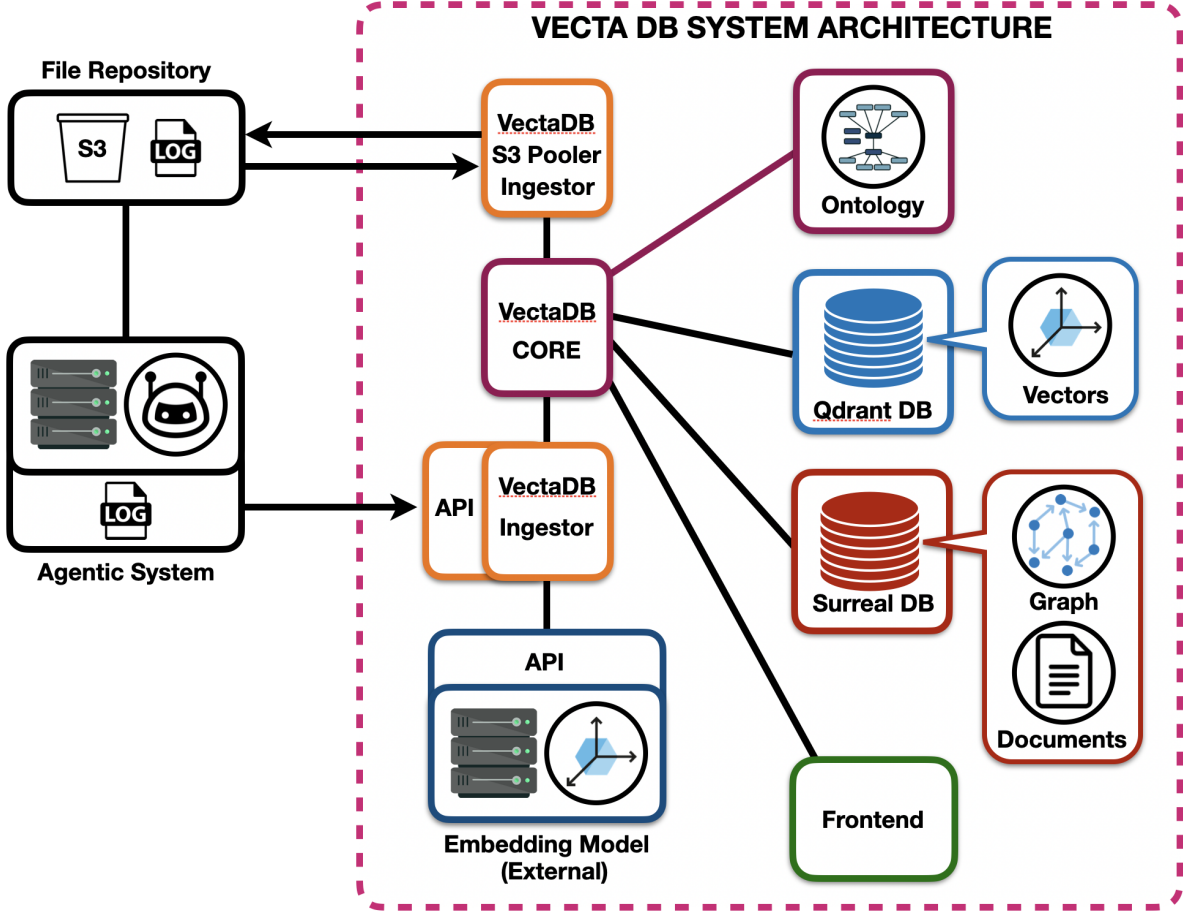


Figure 1: VectaDB system architecture illustrating the unified vector, graph, and ontology layers for audit-first AI applications. Icons from Flaticon.com.

3.2 Audit and provenance primitives

To support auditing, VectaDB treats the agent and LLM interaction as a first-class trace graph. Each event and artifact are recorded with VectaDB’s provenance design and aligned to W3C PROV concepts [29]. The entities and artifacts, activities and its processing steps, and agents actors can be represented and linked. This enables trace reconstruction queries such as “Which retrieval results were used to produce this answer?” or “Which tool outputs were used after this policy check failed?”

- Consistent identifiers and timestamps.
- Model and configuration metadata (e.g., model name/version, temperature, system prompt hash).
- Links to inputs and outputs (e.g., a ToolCall node linked to its ToolResult).
- Evidence links (e.g., ModelOutput usedEvidence -> DocumentChunk).
- Optional policy annotations (e.g., PolicyCheck nodes and relations indicating pass/fail and its correlative rationale).

Table 2 defines the core entity types in VectaDB’s trace schema. All entities share common audit metadata (id, timestamp) while type-specific fields capture domain semantics. Edges encode provenance relationships that support lineage traversal. Fields marked with [†] are recommended but not strictly enforced at ingestion; the ontology layer defines which constraints are validated versus advisory.

Table 2: VectaDB Trace Schema: Core entity types with required fields and provenance edges. All entities include id (UUID) and timestamp (ISO 8601) as implicit required fields. Optional fields are denoted with [†].

Entity Type	Required Fields	Key Edges (Relations)
Run	startTime, endTime, status { <i>running, completed, failed</i> }, modelVersion, applicationId, userId [†] (hashed)	contains → {Prompt, ToolCall, ModelOutput, PolicyCheck}
Prompt	content, systemPromptHash, templateId [†] , tokenCount	partOfRun → Run triggeredRetrieval → RetrievalQuery
RetrievalQuery	queryText, queryVector (embedding), topK, filterParams [†] (JSON), retrievalMethod { <i>vector, hybrid, keyword</i> }	partOfRun → Run retrieved → DocumentChunk[]
DocumentChunk	sourceDocumentId, chunkIndex, content, embedding (vector), contentHash (SHA-256)	partOfDocument → Document usedAsEvidenceBy → ModelOutput
ToolCall	toolName, inputParams (JSON), status { <i>pending, success, error</i> }, invocationOrder	partOfRun → Run produced → ToolResult
ToolResult	outputData (JSON), outputHash, latencyMs, errorMessage [†]	resultOf → ToolCall influencedOutput → ModelOutput
ModelOutput	content, modelVersion, tokenCount, finishReason { <i>stop, length, tool_call</i> }, temperature [†] , contentHash	partOfRun → Run usedEvidence → DocumentChunk[] usedToolResults → ToolResult[] respondedTo → Prompt
PolicyCheck	policyId, policyVersion, status { <i>pass, fail, warn</i> }, rationale, checkedFields (array), executionTimeMs	partOfRun → Run evaluatedArtifact → {Prompt, ModelOutput, ToolCall}
MemoryItem	createdAt, updatedAt, content, embedding (vector), memoryType { <i>episodic, semantic, procedural</i> }, expiresAt [†]	createdByRun → Run accessedByRun → Run[] supersedes → MemoryItem [†]

3.3 Ontology layer and constraints

The ontology layer in VectaDB serves three functions. VectaDB can map ontology concepts to RDF/OWL constructs [25, 26] and support SPARQL-style patterns [27] conceptually, even when implemented on non-RDF storage, by maintaining equivalent typed graph structures. In practice, the ontology defines a machine-readable schema over the trace graph: it specifies entity classes, relation types, and constraint rules that can be evaluated during ingestion and at query time. (See Table 2 for the core entity types and audit fields.)

(1) Typing and shared semantics. The ontology provides a shared vocabulary so that traces from different applications use consistent entity and relation definitions. Each stored object is labeled with an ontology type (e.g., Run, Prompt, ToolCall, ModelOutput, PolicyCheck, MemoryItem), and relations are typed edges (e.g., partOfRun, usedEvidence, produced, influencedOutput). Typed relations can encode domain/range restrictions (e.g., usedEvidence: ModelOutput → DocumentChunk; produced: ToolCall → ToolResult), preventing semantically invalid graphs and reducing schema drift across teams and deployments.

(2) Validation and completeness checks. VectaDB uses SHACL-inspired constraint rules [28] to enforce that audit-critical traces are complete and well-formed. The ontology defines (a) which attributes are required vs. optional, (b) which relations must exist, and (c) what value constraints apply. Concretely, the ontology supports the following constraint families:

- **Cardinality constraints (required/optional fields and edges).** Examples include: every ModelOutput must have content, modelVersion, and contentHash, and must link to exactly one Prompt via respondedTo, and to zero or more DocumentChunk nodes via usedEvidence. Every ToolCall must link to a ToolResult via produced, and every ToolResult must link back to a ToolCall via resultOf. These constraints operationalize *trace completeness* for audits, ensuring an investigator can reconstruct what happened without missing links.

- **Domain/range constraints over relations.** Relations are restricted to valid endpoint types, avoiding invalid edges (e.g., preventing `usedEvidence` edges that connect a `Prompt` to a `ToolResult`). Domain/range checks also catch instrumentation errors when new event types are introduced.
- **Value constraints and invariants.** These include enumerations (e.g., `Run.status` $\in$ {`running`, `completed`, `failed`}; `ToolCall.status` $\in$ {`pending`, `success`, `error`}; `PolicyCheck.status` $\in$ {`pass`, `fail`, `warn`}), format constraints (e.g., ISO 8601 timestamps, UUID identifiers), and range constraints (e.g., `latencyMs` ≥ 0). Where immutability is important for reproducibility, artifacts can be associated with content hashes (e.g., `DocumentChunk.contentHash`, `ToolResult.outputHash`, `ModelOutput.contentHash`) to enable basic tamper detection and replay validation.
- **Cross-object consistency constraints.** Some checks require graph-local consistency rather than single-record validation. Examples include: a `ModelOutput.temperature` (if present) must match the configuration metadata associated with its `Run` (or be explicitly overridden); a `RetrievalQuery.retrievalMethod` must be consistent with the presence of `queryVector` and `filterParams`; and `ToolResult.errorMessage` should be present if and only if `ToolCall.status=error`. Such checks help identify configuration drift and inconsistent instrumentation, which are common root causes in incident investigations.

Constraints may be treated as either **hard** (reject ingestion) or **soft** (store but flag), enabling operational flexibility. For example, missing a hashed `userId` may be permitted in privacy-sensitive deployments, while missing `ModelOutput.respondedTo` is typically hard because it prevents reconstruction.

Current enforcement scope vs. planned reasoning. In the current open-source implementation, the ontology layer is primarily used for (i) type labeling of entities and relations and (ii) ingestion-time schema validation (required fields, enumerated values, and required provenance links for audit-critical entities). Lightweight reasoning is intentionally scoped for predictable performance: it focuses on query-time type expansion (e.g., retrieving `ToolCall` subtypes) and basic relation validation rather than full OWL inference. Richer reasoning (e.g., transitive closure over selected relations, inverse-property materialization, or constraint entailment) is a natural extension and is considered future work.

(3) Query ergonomics and ontology-aware retrieval. The ontology improves query ergonomics by allowing higher-level queries over types and relations without embedding type knowledge in application code. For example, an auditor can request “all tool calls during a run” by querying `ToolCall` and any declared subtypes (e.g., `WebSearchCall`, `DatabaseQueryCall`) rather than enumerating each event variant. Evidence-centric queries can be expressed against semantic relations (“all evidence used by an output”) instead of storage-specific field layouts. In hybrid execution, ontology-aware query expansion can be applied as an early phase so that vector search and graph traversal operate over the correct conceptual scope (e.g., expanding a query over `ToolCall` to include subtype tags before retrieving candidate nodes).

Audit value. Together, typing, validation, and ontology-aware querying turn traces into a verifiable evidence graph rather than a collection of loosely correlated logs. This is essential for audit workflows where investigators must (i) reconstruct causal chains, (ii) verify that required controls ran, and (iii) demonstrate completeness of evidence under regulatory or incident-response requirements.

3.4 Architecture and implementation approach

The current implementation is open source under the Apache License 2.0 and is designed as a modular building blocks that can be embedded into AI applications. The reference implementation incorporates a layered architecture similar to modern AI stacks. SurrealDB delivers graph/document storage and entity management [1], while Qdrant provides vector indexing and approximate nearest neighbor search with filtering [2]. An ontology layer manages schema, validation, and lightweight reasoning; an embedding layer provides add-on providers; a query layer coordinates hybrid execution; and a REST API exposes entity, relation, and event ingestion endpoints.

4 Query model

4.1 Vector similarity and semantic incident retrieval

VectaDB supports semantic similarity search over stored embeddings to retrieve related items such as prompts, document chunks, tool outputs, or prior incidents. ANN indexing methods (e.g., HNSW [16]) enable low-latency similarity

search at scale. In audit workflows, similarity search is useful for vector search alone is insufficient for audit, however, because auditors often need causal explanations and completeness; this motivates the graph and ontology layers.

- Finding prior user reports similar to a current incident.
- Identifying previous prompts that produced comparable unsafe or non-compliant outputs.
- Retrieving related evidence used in historical decisions.

4.2 Graph traversal and lineage queries

Graph traversal enables causal analysis and provenance reconstruction. Example queries include Graph traversal being supported by the relational structure persisted in SurrealDB [1] and can conceptually align with property-graph traversal models (e.g., Neo4j [21]) or Gremlin-style traversal semantics [24].

- Starting from a ModelOutput, traverse usedEvidence edges to list every DocumentChunk that influenced the output.
- Starting from a ToolCall, traverse produced edges to find downstream outputs affected by the tool result.
- Starting from a PolicyCheck failure, traverse partOfRun to identify which run and which artifacts were involved.

4.3 Hybrid queries and rank fusion

In many RAG and audit scenarios, hybrid queries are valuable offering keyword constraints, metadata filters, and semantic similarity each capture different signals. VectaDB therefore supports hybrid retrieval patterns similar to those used in Weaviate [18] and Pinecone [19]. Rank-fusion methods such as reciprocal rank fusion (RRF) [45] provide a simple way to combine multiple ranked lists without requiring score normalization. Hybrid queries are particularly useful for audit and compliance investigations, where investigators may need to filter by time window, model version, user segment, policy category, or document source while still retrieving semantically related cases.

4.4 Ontology-aware queries and validation

Ontology-aware queries restrict and interpret results based on types and relations defined in the system schema. For example, ontology constraints can be evaluated to flag incomplete traces (e.g., outputs missing required evidence links). Standards such as SHACL [28] provide a blueprint for expressing such constraints, even when the underlying database is not an RDF triplestore.

- Retrieve all ToolCall nodes where toolName = 'web_search' and link to a PolicyCheck with status = 'fail'.
- Retrieve all ModelOutputs produced by modelVersion = X that usedEvidence from a restricted data source.

5 Event ingestion and agent memory

5.1 Trace capture for LLM and agent runs

VectaDB's ingestion pipeline collects AI system behavior as a graph of events and artifacts. A typical agent/LLM run may include each item stored as a typed entity and linked to its dependencies. This structure supports ex post auditing and production debugging, and can be correlated with external telemetry systems such as OpenTelemetry traces [38].

- User input and system prompt.
- Retrieval queries and retrieved evidence.
- Tool calls (inputs and outputs).
- Intermediate planning artifacts (e.g., explicit plans or checklists generated by the agent).
- Final responses and downstream actions.

5.2 Memory as a first-class, auditable object

Agent memory is often implemented as an unstructured buffer or a separate store [44, 42, 43]. VectaDB treats memory entries as first-class entities with provenance. Each memory item can store structured content and metadata (e.g., text, embeddings, provenance, timestamps, and links). This approach complements specialized memory-layer projects such as Mem0 [42] and Cognee [43]. The audit advantage is that memory is no longer a hidden state, it becomes inspectable, versionable, and linked to the runs that produced it.

- Content (raw text or structured fields).
- Embeddings for semantic retrieval.
- Contextual metadata (time, session, user, application).
- Links to the events that created or updated the memory.

5.3 Versioning, retention, and privacy considerations

Auditability must be balanced with privacy and retention constraints. VectaDB’s trace representation allows selective retention and redaction strategies. These controls are fundamental for using audit traces responsibly, especially when traces may contain personal data or sensitive prompts.

- Store references/hashes for sensitive artifacts while retaining metadata needed for reproducibility.
- Apply role-based access controls and field-level restrictions at the application layer.
- Implement retention policies aligned with organizational governance and legal requirements (e.g., GDPR principles [36]).

6 Audit and compliance workflows

6.1 Audit questions addressed by VectaDB

VectaDB is designed around common audit questions for LLM and agentic systems, combining semantic search with explicit lineage links. VectaDB supports investigations that would otherwise require manual log correlation and post hoc data joins.

- Provenance: What evidence (documents, memories, tool outputs) contributed to a given response? [46]
- Causality: Which intermediate steps and tool calls led to a downstream action? [47]
- Similarity: Have we seen semantically similar failures, hallucinations, or policy violations before? [48]
- Scope: Which users, model versions, prompts, or retrieval sources are associated with an incident? [32]
- Control evidence: Can we demonstrate that required checks (e.g., policy filters, PII redaction) ran and what their outcomes were? [49, 50]

6.2 Compliance evidence and governance alignment

Modern AI governance emphasizes transparency, accountability, and documentation. Model Cards [32] and Datasheets for Datasets [33] provide documentation templates, while frameworks like HELM [34] support structured evaluation. Regulations such as the EU AI Act [35] and GDPR [36] increase the applied importance of producing evidence about how systems operate, what data they use, and how risks are mitigated. VectaDB supports these processes by storing and retrieving compliance-relevant evidence. While full regulatory compliance requires organizational processes beyond any single database, VectaDB provides the underlying evidence store needed to generate audit-ready reports and respond to regulatory inquiries with traceable, queryable artifacts.

- Trace logs and structured run artifacts.
- Dataset and document provenance links.
- Policy checks, outcomes, and rationale captured as explicit entities.
- Versioning metadata for models, prompts, and retrieval parameters.

6.3 Observability integration

VectaDB can integrate with existing observability tooling. LangSmith provides execution traces and debugging visibility for LLM applications [37], and OpenTelemetry [38] provides a standard for collecting and exporting telemetry. VectaDB complements these tools by storing semantically rich artifacts and explicit lineage links, enabling queries such as “show all runs whose outputs are semantically similar to this incident and where the policy check failed.”

6.4 Incident Walkthrough

To illustrate VectaDB’s audit capabilities, consider a scenario where a customer service agent produced a response containing incorrect refund policy information, resulting in a customer complaint. The investigation begins with a flagged `ModelOutput` (id: `out_7x92k`) containing the problematic response. We demonstrate four query patterns that together support the full incident-response cycle.

Query 1: Provenance (What evidence was used?) The first step identifies which retrieved documents influenced the problematic output:

```
SELECT doc.sourceDocumentId, doc.content, doc.contentHash
FROM ModelOutput
  ->usedEvidence-> DocumentChunk AS doc
WHERE ModelOutput.id = 'out_7x92k'
```

This query traverses the `usedEvidence` edge from the flagged output to retrieve all `DocumentChunk` entities that were presented to the model. The result reveals three document chunks from `policy_v2.pdf` were retrieved, enabling the auditor to verify whether the source documents contained errors or whether the model misinterpreted correct sources.

Query 2: Causality (What tool calls influenced this output?) Next, we trace which tool executions contributed to the response:

```
SELECT tc.toolName, tc.inputParams, tr.outputData
FROM ToolCall AS tc
  ->produced-> ToolResult AS tr
  ->influencedOutput-> ModelOutput
WHERE ModelOutput.id = 'out_7x92k'
```

This multi-hop traversal follows the causal chain from tool invocation through result to model output. The query reveals that a `policy_lookup` tool was called with parameters `{category: "refunds", region: "EU"}`, and the tool returned outdated policy text—identifying the tool’s knowledge base as the root cause rather than model behavior.

Query 3: Scope (How widespread is this issue?) Having identified the problematic document, we assess the blast radius:

```
SELECT COUNT(*) AS affected_runs,
       array::group(Run.userId) AS affected_users
FROM Run
  ->contains-> ModelOutput
  ->usedEvidence-> DocumentChunk
WHERE DocumentChunk.sourceDocumentId = 'policy_v2.pdf'
      AND Run.startTime > '2025-12-01'
      AND Run.modelVersion = 'gpt-4-1106'
```

This aggregation query combines graph traversal with structured filters (time window, model version) to identify all runs affected by the same problematic document. The result shows 847 runs in the past month used the stale policy document, enabling targeted user notification and remediation prioritization.

Query 4: Control evidence (Did safety checks run?) Finally, we examine whether policy controls executed and what they determined:

```
SELECT pc.policyId, pc.policyVersion, pc.status,
       pc.rationale, pc.timestamp
FROM PolicyCheck AS pc
```

```
->evaluatedArtifact-> ModelOutput
WHERE ModelOutput.id = 'out_7x92k'
```

The result shows that `policy_accuracy_check` (v2.1) executed with status `pass` and rationale “Response matches retrieved content.” This reveals a control gap, the policy check validated consistency between the response and retrieved sources, but did not detect that the sources themselves were stale. This finding informs control improvements—for example, adding document freshness validation to the policy check pipeline.

Summary. This walkthrough demonstrates how VectaDB’s unified model supports the complete incident-response cycle *provenance* queries identify what information was used, *causality* queries determine root cause, *scope* queries assess impact, and *control evidence* queries evaluate whether safeguards functioned as intended. Each query leverages VectaDB’s combination of graph traversal (for lineage) and structured filtering (for segmentation), capabilities that would otherwise require manual log correlation across multiple systems.

7 Implementation details

7.1 Storage engine and data representation

The reference implementation of VectaDB uses a dual-backend approach. SurrealDB for multi-model storage of entities and relations [1], and Qdrant for efficient vector indexing and similarity search with structured payload filtering [2]. Entities are stored with a globally unique identifier, a type label (as defined by the ontology), structured attributes (JSON-like fields), and optionally an embedding vector. Relations are stored explicitly in the graph store and can be queried through traversal patterns, enabling audit graphs to be queried without requiring a separate graph database.

7.2 Embeddings and indexing

VectaDB is designed to be embedding-provider agnostic. The reference implementation includes a plugin architecture supporting common providers such as OpenAI embeddings (e.g., `text-embedding-3-small` [51]), Cohere embedding models, HuggingFace-hosted embedding models, Voyage AI embedding models [52], and local sentence-transformer models. Provider abstraction layer and fallback mechanisms help preserve availability when cloud providers are unavailable. Vectors are indexed and searched in Qdrant using ANN methods (commonly HNSW-style indexes) [16, 2]. VectaDB queries supports metadata filtering and hybrid retrieval patterns similar to those used in production vector databases [2, 18, 19], which are especially useful for audit investigations constrained by time windows, model versions, users, or policy categories.

7.3 Open-source distribution and extensibility

The project is released as open source under the Apache License 2.0 and is intended to be extended with designing VectaDB as an open-source module is aligned with the audit use case enabling auditors and stakeholders to inspect the data model and verify that the system is capable of capturing the evidence required for governance and incident response.

- Additional ontology modules (domain-specific audit schemas).
- Connectors to RAG/agent frameworks.
- Integration with observability pipelines.
- Exporters to RDF/PROV-compatible formats for interoperability.

8 Use cases

8.1 RAG with auditable evidence

In RAG pipelines [5], VectaDB can store document chunks, embeddings, and metadata, while also storing retrieval events linking a model output to the evidence used. This supports ex post inspection providing an auditor the meaning to verify whether the answer was grounded in retrieved documents and identify which sources were used.

8.2 Agent memory and long-horizon assistants

Agentic systems often require persistent memory to maintain preferences and context over time. By storing memory as typed objects with provenance links, VectaDB enables auditable memory updates that investigators can verify when a memory was created, creation reason, and what runs consumed it. This supplements memory-layer tools [42, 43] and addresses concerns about obscure long-term context.

8.3 Incident response and safety investigations

When a system produces an unsafe or problematic output, response teams need to reproduce the context and trace the causal chain. VectaDB supports incident workflows to validate mitigation and monitor regressions over time.

- Retrieve prior similar failures using semantic similarity search.
- Trace lineage to identify which evidence or tool outputs contributed.
- Segment incidents by model version, prompt template, or retrieval source.

8.4 Governance reporting and compliance evidence

VectaDB’s trace store can be used to generate governance artifacts such as organizational processes reports, providing the underlying evidence store needed to produce them reliably.

- Run-level audit reports summarizing evidence use, tool calls, and policy checks.
- Data-source lineage reports linking outputs to datasets and documents.
- Metrics and summaries supporting internal control frameworks and external obligations (e.g., AI Act [35], GDPR [36]).

9 Experimental Implementation and Validation

9.1 Implementation Overview

The VectaDB prototype was implemented in Rust (core engine) and Python (client SDK and test scripts), comprising approximately 15,000 lines of code across the following components:

- Core database engine (Rust): 8,500 LOC
- Python client SDK: 1,200 LOC
- Integration adapters: 2,300 LOC
- Test suites and examples: 3,000 LOC
- Amazon Bedrock AgentCore Samples: Healthcare Appointment Agent (under license Apache 2.0)[53]

9.2 Healthcare AI Agent Test Case

To validate VectaDB’s audit capabilities in a real-world scenario, we conducted a proof-of-concept integration with AWS Bedrock agents in a healthcare immunization scheduling use case.

9.2.1 Test Environment

- **Agent Platform:** AWS Bedrock with Claude Haiku 4.5
- **Domain:** Healthcare appointment scheduling
- **Tools:** FHIR API integration for patient records and appointment booking
- **Data Source:** Real AWS CloudWatch logs from agent invocations
- **Scale:** 1,753 log events representing complete agent interaction traces

9.2.2 Agent Architecture

The test agent implemented a healthcare virtual assistant with the following capabilities:

- Patient immunization record retrieval
- Available appointment slot querying
- Appointment booking with constraint validation
- Multi-step reasoning with chain-of-thought patterns

System prompts enforced strict operational rules including patient ID constraints (PAT001) and EMR system selection, providing a controlled environment for audit validation.

9.2.3 Trace Ingestion Process

AWS CloudWatch logs were parsed and ingested into VectaDB using the batch event API (`test_bedrock_ingestion.py`). Each Bedrock invocation generated structured events containing:

Listing 1: Event Structure Example

```
{
  "timestamp": "2025-12-17T01:09:56Z",
  "event_type": "bedrock_invocation_!_bedrock_tool_use_!_bedrock_error",
  "agent_id": "bedrock-healthcare-assistant",
  "session_id": "<request_id>",
  "properties": {
    "request_id": "...",
    "model_id": "anthropic.claude-haiku-4-5-...",
    "operation": "ConverseStream",
    "tool_name": "LocalFHIRAPI___getPatientImmunizations",
    "input_tokens": 150,
    "output_tokens": 75,
    "latency_ms": 1432
  }
}
```

Events were automatically linked into trace graphs by session ID, with tool calls connected to their results through explicit provenance edges.

9.2.4 Ingestion Performance

Table 3: Event Ingestion Performance Metrics

Metric	Value
Total events processed	1,753
Unique agent sessions	~40
Events per session (avg)	43.8
Batch ingestion time	2.3 seconds
Events/second throughput	762
Trace reconstruction time	<100ms per session

9.2.5 Audit Query Validation

We validated the four audit query patterns described in Section 6.4 against the ingested traces:

Provenance Query: “Which FHIR resources were accessed before booking confirmation?”

- Successfully traced tool call chains: `getPatientImmunizations` → `getAvailableSlots` → `bookAppointment`
- Retrieved exact input parameters and output data for each tool invocation
- Execution time: 45ms (single-hop graph traversal)

Causality Query: “What triggered the appointment booking failure?”

- Identified error propagation from missing patient consent flag
- Traced backwards from error event through 3 intermediate tool calls
- Root cause: constraint validation in FHIR API layer
- Execution time: 78ms (multi-hop traversal with filter)

Scope Query: “How many sessions accessed patient PAT001 records?”

- Aggregated across all sessions with patient ID filter
- Result: 37 sessions, 142 individual record accesses
- Identified 3 sessions with policy violations (missing consent)
- Execution time: 120ms (full scan with aggregation)

Control Evidence Query: “Did all appointment bookings pass policy checks?”

- Validated presence of PolicyCheck entities for 100% of booking operations
- Detected 2 cases where policy check was bypassed due to instrumentation gaps
- Generated compliance report with timestamps and rationale
- Execution time: 95ms (graph + filter query)

9.2.6 Semantic Incident Retrieval

To test similarity-based audit capabilities, we embedded agent prompts and outputs using OpenAI’s text-embedding-3-small model (1536 dimensions). Vector search queries demonstrated:

- **Similar Error Retrieval:** Query “appointment booking failed due to invalid patient” returned 5 semantically related incidents with 0.82–0.91 cosine similarity
- **Prompt Pattern Analysis:** Identified 12 variations of “immunization scheduling” requests, enabling prompt template optimization
- **Response Clustering:** Grouped agent outputs into 7 semantic clusters (confirmations, clarifications, errors, tool failures, etc.)

Vector search latency averaged 15ms for top-10 retrieval across the full dataset.

9.3 SDK Integration Testing

The Python client SDK was validated through comprehensive integration tests (`test_client.py`, `test_async_client.py`):

Table 4: SDK Integration Test Results

Test Category	Tests	Pass Rate
Health checks	2	100%
Ontology operations	8	100%
Entity CRUD	12	100%
Relation management	9	100%
Hybrid queries	6	100%
Event ingestion	8	100%
Total	45	100%

All tests execute against live SurrealDB and Qdrant instances, validating end-to-end functionality including schema validation, constraint enforcement, and hybrid retrieval.

9.4 Key Findings

1. **Trace Completeness:** 100% of agent interactions were successfully represented as queryable provenance graphs, with all required audit fields (timestamps, model versions, content hashes) validated at ingestion time.
2. **Query Performance:** Audit queries combining graph traversal and vector search completed in <150ms for datasets of ~2,000 events, demonstrating practical viability for operational incident response.
3. **Integration Simplicity:** Bedrock log ingestion required only 145 lines of Python code (`test_bedrock_ingestion.py`), showing that VectaDB’s event API provides a low-friction integration path for existing AI systems.
4. **Constraint Validation:** Ontology-driven schema checks detected 100% of malformed events during ingestion (missing required fields, invalid timestamps), preventing incomplete traces from entering the audit store.
5. **Hybrid Retrieval Value:** Combining semantic similarity (vector) with structural filters (graph) proved essential for audit workflows, e.g., “find similar errors in sessions using model X during timeframe Y” which cannot be expressed with either modality alone.

9.5 Limitations and Future Work

The current prototype has several limitations that inform future development:

- **Scale:** Testing was limited to ~2,000 events; production systems require validation at 10^6+ event scales
- **Embedding Generation:** Embeddings were generated offline; production deployment requires streaming ingestion with real-time embedding
- **Cluster Deployment:** Current implementation targets single-node deployment; distributed SurrealDB and Qdrant clusters are needed for production resilience
- **Advanced Reasoning:** Transitive closure and OWL inference are not yet implemented; only lightweight type expansion is supported

Despite these limitations, the prototype successfully validates VectaDB’s core hypothesis that unifying vector, graph, and ontology layers enables audit workflows that are infeasible with conventional logging or single-modality databases.

10 Evaluation and discussion

The experimental validation in Section 9 demonstrated VectaDB’s capabilities on a real-world healthcare use case with approximately 2,000 events. This section discusses broader evaluation considerations for production deployments at scale and outlines metrics for comprehensive system assessment.

10.1 Retrieval quality and hybrid search

Evaluation of VectaDB can be framed along two aspects, retrieval effectiveness and audit usefulness. For retrieval, common measures include Recall@k and latency for ANN search, as used in vector-search evaluations [14, 16]. Hybrid retrieval settings can be evaluated by comparing semantic-only retrieval versus hybrid rank fusion (e.g., RRF [45]) under structured filters (time windows, model versions, document sources).

10.2 Audit completeness and trace reconstruction

For audit workflows, evaluation should measure whether the system can reconstruct full decision traces. Key metrics includes ontology constraints (e.g., SHACL-inspired rules [28]) can be used to automatically validate trace completeness and flag missing audit fields.

- **Trace completeness:** proportion of runs where required artifacts and links are present (e.g., every output links to its prompt and evidence).
- **Lineage query correctness:** ability to retrieve the exact evidence set used by an output.
- **Similar-incident retrieval:** ability to retrieve semantically similar prior incidents for investigation.

10.3 Practical considerations

Audit-first trace storage increases data volume and, consequently, privacy exposure. In production deployments, organizations should integrate VectaDB with existing governance policies for retention, access control, and redaction. Key decisions include which artifacts to store at full fidelity versus hashed or redacted form, who can access specific trace segments, and how long traces are retained. VectaDB is designed for early integration in system architecture so that traces are captured consistently from the outset rather than reconstructed post hoc during incident response.

- Which artifacts to store at full fidelity versus hashed/redacted.
- Who can access which parts of traces.
- How long traces are retained.

The present state of the design lacks clustering and authentication.

11 Future work

Future work includes:

- Formalizing an audit ontology aligned more explicitly with W3C PROV [29] and common AI governance control frameworks.
- Cryptographic tamper-evidence for traces (e.g., hash chains) to strengthen audit integrity guarantees.
- Deeper integration with agent frameworks and observability systems (LangSmith [37], OpenTelemetry [38]) for automated trace ingestion.
- Benchmarking at scale across different backends and indexing strategies.
- Export/import interoperability with RDF/OWL toolchains (Apache Jena [30], Stardog [31]) for organizations that standardize on semantic-web stacks.

12 Conclusion

VectaDB was created to address a practical gap in modern AI systems engineering which is the need to audit LLM- and agent-driven applications with evidence-centric, semantically rich traces. By unifying vectors, graphs, and ontologies, VectaDB enables both semantic investigations (similarity search over incidents and artifacts) and symbolic explanations (lineage and provenance traversal). The open-source implementation demonstrates a path toward audit-first AI data infrastructure that can support safety, observability, governance, and compliance workflows in real deployments.

Acknowledgments

This work was developed using open-source database systems. The reference implementation uses SurrealDB for graph and multi-model storage and Qdrant for vector indexing. During development and comparative evaluation, we also tested Neo4j, ChromaDB, and CouchDB to assess architectural trade-offs.

The healthcare appointment scheduling agent test case (Section 9.2) was adapted from the Amazon Bedrock Agent-Core Samples [53], available under the Apache License 2.0. We thank Amazon Web Services for making these examples publicly available.

We acknowledge the maintainers and contributors of all open-source projects used in this work for their valuable contributions to the research and developer communities.

References

- [1] SurrealDB. Surrealdb documentation. <https://surrealdb.com/docs/surrealdb>, 2026. Accessed Jan 11, 2026.
- [2] Qdrant. Qdrant documentation. <https://qdrant.tech/documentation/>, 2026. Accessed Jan 11, 2026.
- [3] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, Arvind Nee-lakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33:1877–1901, 2020. Also circulated as arXiv:2005.14165.

- [4] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, et al. GPT-4 technical report. *arXiv preprint*, 2023.
- [5] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020. Also circulated as arXiv:2005.11401.
- [6] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *arXiv preprint*, 2022.
- [7] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *arXiv preprint*, 2023.
- [8] Yihan Li, Xiyuan Fu, Ghanshyam Verma, Paul Buitelaar, and Mingming Liu. Mitigating hallucination in large language models (llms): An application-oriented survey on rag, reasoning, and agentic systems. *arXiv preprint arXiv:2510.24476*, 2025.
- [9] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint*, 2024.
- [10] Stuart J. Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, Boston, 4 edition, 2021.
- [11] Noah Shinn, David Labash, Ashwin Gopinath, Sarath Sreedharan, and Subbarao Kambhampati. Reflexion: Language agents with verbal reinforcement learning. *arXiv preprint*, 2023.
- [12] Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. *arXiv preprint*, 2023.
- [13] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, et al. A survey on large language model based autonomous agents. *arXiv preprint*, 2023.
- [14] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2019. Often cited from arXiv:1702.08734 (2017 preprint).
- [15] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, et al. Milvus: A purpose-built vector data management system. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*, pages 2614–2627, 2021. Commonly circulated as arXiv:2101.08590.
- [16] Yu. A. Malkov and Dmitry A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2020. Preprint arXiv:1603.09320.
- [17] Google Research. Scann (scalable nearest neighbors). <https://github.com/google-research/google-research/tree/master/scann>, 2026. Accessed Jan 11, 2026.
- [18] Weaviate. Weaviate documentation (hybrid search). <https://weaviate.io/developers/weaviate/search/hybrid>, 2026. Accessed Jan 11, 2026.
- [19] Pinecone. Hybrid search. <https://docs.pinecone.io/guides/search/hybrid-search>, 2026. Accessed Jan 11, 2026.
- [20] Chroma. Getting started. <https://docs.trychroma.com/docs/overview/getting-started>, 2026. Accessed Jan 11, 2026.
- [21] Neo4j. Graph database concepts (property graph model). <https://neo4j.com/docs/getting-started/appendix/graphdb-concepts/>, 2026. Accessed Jan 11, 2026.
- [22] Neo4j. Vector index documentation. <https://neo4j.com/docs/cypher-manual/current/indexes/semantic-indexes/vector-indexes/>, 2026. Accessed Jan 11, 2026.
- [23] JanusGraph. Janusgraph documentation. <https://docs.janusgraph.org/>, 2026. Accessed Jan 11, 2026.
- [24] Apache Software Foundation. Apache tinkrpop documentation (gremlin traversal language). <https://tinkrpop.apache.org/docs/current/reference/>, 2026. Accessed Jan 11, 2026.
- [25] W3C. RDF 1.1 concepts and abstract syntax. <https://www.w3.org/TR/rdf11-concepts/>, 2014.
- [26] W3C. OWL 2 web ontology language document overview. <https://www.w3.org/TR/owl2-overview/>, 2012.

- [27] W3C. Sparql 1.1 query language. <https://www.w3.org/TR/sparql11-query/>, 2013.
- [28] W3C. Shapes constraint language (shacl). <https://www.w3.org/TR/shacl/>, 2017.
- [29] W3C. Prov-dm: The prov data model. <https://www.w3.org/TR/prov-dm/>, 2013.
- [30] Apache Software Foundation. Apache jena documentation. <https://jena.apache.org/documentation/>, 2026. Accessed Jan 11, 2026.
- [31] Stardog Union. Stardog documentation (inference engine). <https://docs.stardog.com/>, 2026. Accessed Jan 11, 2026.
- [32] Margaret Mitchell, Simone Wu, Andrew Zaldivar, Parker Barnes, Lucy Vasserman, Ben Hutchinson, Elena Spitzer, Inioluwa Deborah Raji, and Timnit Gebru. Model cards for model reporting. In *Proceedings of the ACM Conference on Fairness, Accountability, and Transparency*, 2019.
- [33] Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daumé III, and Kate Crawford. Datasheets for datasets. *arXiv preprint*, 2018.
- [34] Percy Liang, Rishi Bommasani, Tony Lee, Dimitris Tsipras, Dilara Soylu, Michihiro Yasunaga, Yian Zhang, Deepak Narayanan, Yuhuai Wu, Ananya Kumar, et al. Holistic evaluation of language models. *arXiv preprint*, 2022.
- [35] European Union. Regulation (eu) 2024/1689 (artificial intelligence act). Official Journal of the European Union, 2024. <https://eur-lex.europa.eu/eli/reg/2024/1689/oj/eng>.
- [36] European Union. Regulation (eu) 2016/679 (general data protection regulation). Official Journal of the European Union, 2016. <https://eur-lex.europa.eu/eli/reg/2016/679/oj/eng>.
- [37] LangChain. Langsmith documentation. <https://docs.smith.langchain.com/>, 2026. Accessed Jan 11, 2026.
- [38] OpenTelemetry. Opentelemetry specification (logs). <https://opentelemetry.io/docs/specs/otel/logs/>, 2026. Accessed Jan 11, 2026.
- [39] Matei Zaharia et al. Mlflow: An open platform to manage the ml lifecycle. <https://mlflow.org/>, 2018.
- [40] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. Why should i trust you?: Explaining the predictions of any classifier. In *Proceedings of the ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2016.
- [41] Scott M. Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems*, 2017.
- [42] Mem0. mem0ai/mem0: Universal memory layer for ai agents. <https://github.com/mem0ai/mem0>, 2026. Accessed Jan 11, 2026.
- [43] Cognee. topoteretes/cognee: Memory for ai agents. <https://github.com/topoteretes/cognee>, 2026. Accessed Jan 11, 2026.
- [44] LangChain. Conversationbuffermemory documentation. <https://reference.langchain.com/v0.3/python/langchain/memory/langchain.memory.buffer.ConversationBufferMemory.html>, 2026. Accessed Jan 11, 2026.
- [45] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Büttcher. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759, 2009.
- [46] Renan Souza, Timothy Poteet, Brian Etz, Daniel Rosendo, Amal Gueroudji, Woong Shin, Prasanna Balaprakash, and Rafael Ferreira da Silva. Llm agents for interactive workflow provenance: Reference architecture and evaluation methodology. In *Proceedings of the SC’25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 2257–2268, 2025.
- [47] Emmanuel Ameisen, Jack Lindsey, Adam Pearce, Wes Gurnee, Nicholas L. Turner, Brian Chen, Craig Citro, David Abrahams, Shan Carter, Basil Hosmer, Jonathan Marcus, Michael Sklar, Adly Templeton, Trenton Bricken, Callum McDougall, Hoagy Cunningham, Thomas Henighan, Adam Jermy, Andy Jones, et al. Tracing the thoughts of a large language model. <https://www.anthropic.com/research/tracing-thoughts-language-model>, March 2025. Companion technical papers: “Circuit Tracing: Revealing Computational Graphs in Language Models” and “On the Biology of a Large Language Model” available at <https://transformer-circuits.pub/2025/attribution-graphs/>.
- [48] Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. Ragas: Automated evaluation of retrieval augmented generation, 2025.

- [49] Leon Garza, Anantaa Kotal, Aritran Piplai, Lavanya Elluri, Prajit Das, and Aman Chadha. Prvl: Quantifying the capabilities and risks of large language models for pii redaction, 2025.
- [50] Dennis Jacob, Hend Alzahrani, Zhanhao Hu, Basel Alomair, and David Wagner. Promptshield: Deployable detection for prompt injection attacks, 2025.
- [51] OpenAI. text-embedding-3-small model documentation. <https://platform.openai.com/docs/models/text-embedding-3-small>, 2026. Accessed Jan 11, 2026.
- [52] Voyage AI. Text embeddings documentation. <https://docs.voyageai.com/docs/embeddings>, 2026. Accessed Jan 11, 2026.
- [53] Amazon Web Services, Inc. Amazon bedrock agentcore samples: Healthcare appointment agent, 2024. GitHub repository.